n = 83: round 2 of the root-index build speedup, exact changes only (Tier 1 design)

2026-10-20

n = 83: round 2 of the root-index build speedup, exact changes only (Tier 1 design)

Status (2026-10-20): design frozen, run 39464544962 launched; no result yet. This round makes only exact, byte-identical changes. It uses no sampled self-test. Every profile figure and saving below comes from the shared cloud-agent VM. They are local diagnostics, used only to rank the candidates and size the design. The Tier 1 run on one GitHub runner is the only evidence this experiment will produce.

Why this run

The n = 83 a = 1 baseline row is the **index-build producer**:

- experiment `koblitz-n83-indexbuild-20261020`;
- PR #1006, run 39383739494;
- source `bc6a31c7...`, binary `ic_online_indexbuild_r770`;
- Tier 1-qualified at 1.7406 [1.7129, 1.7676] over the closure + index-lift row.

In that run the row (its side B) spent its 14.21 s end to end as follows:

Stage (parent run, side B means)	Time (ms)	Share of E2E
root-index build	7,247.2	51%
rank stage	4,429.9	31%
full scan-order self-test	1,531.6	11%
online (one target)	586.7	4%
rest of setup (base load, basis, tables, input control)	about 416	3%

The index build is still the largest stage. The rank stage is the next largest, but it is not an index-build or setup cost, so this round leaves it alone (see Not in scope).

1. Profile of the current build (local diagnostic)

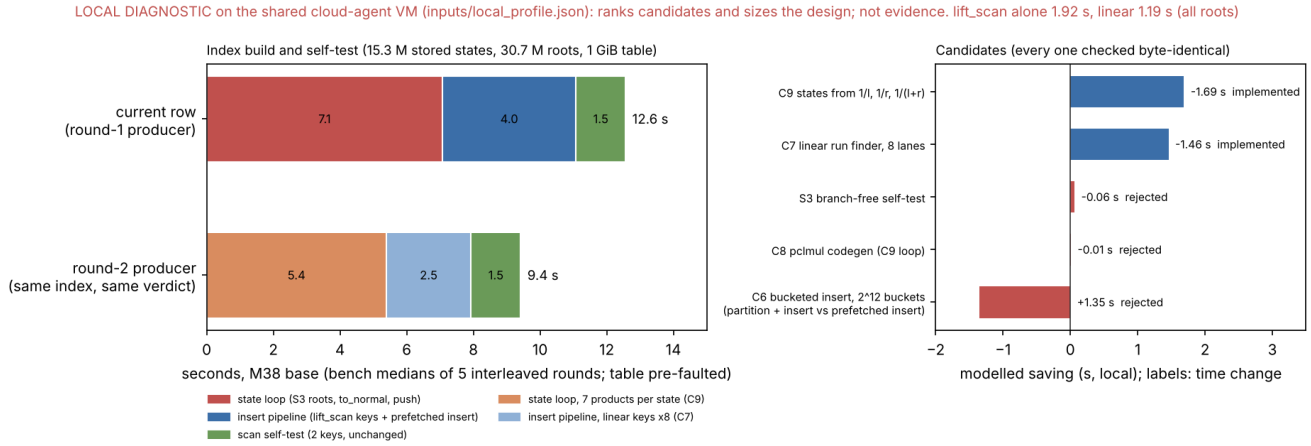


Figure 1: LOCAL DIAGNOSTIC: the row's index build and self-test on the M38 closure header against the round-2 build (bench medians of 5 interleaved rounds), and every candidate with its modelled saving and status. Rendered from `inputs/local_profile.json` by `make_figures.py` (svg, pdf).

Method. This follows round 1's instrumented-scratch approach. `profile/profile2_patch.py` patches a *scratch copy* of the row's producer, which is never committed as a producer. The patched process loads the M38 header exactly as the producer does and then does two things.

- With `KIC_INDEX_PROFILE2=1`, it builds the producer's index with the producer's own functions, then times each component alone and every candidate. Each candidate is asserted equal to the producer's output.
- With `KIC_INDEX_PROFILE2=bench`, it runs five interleaved rounds in rotated order. Each round times the insert pipelines on one pre-faulted, re-zeroed table (so that first-touch faults drop out of the comparison), then the state loops, then the self-tests.

Each process was pinned with `taskset -c 2` and built with rustc 1.97 (the workflow pins 1.99). `perf` is not available. The VM's transparent huge pages are `madvise`, but the parent's runner had `always`, so the local first-touch fault costs (about 0.9 s for the 1 GiB table) are much larger than on the runner. That is one reason the bench excludes them. `profile/summarize2.py` writes `inputs/local_profile.json` from `profile/raw2/`.

Component of the row's index build (M38: 15,341,360 stored states, 30,682,720 roots, 2^26 slots)	Local (ms)
state loop (bench median of 5)	**7,074**
prepare (2 table squares, 2 products)	1,174
Montgomery products + one inversion per row	1,017
finish (6 products, half trace, trace test)	1,858
`to_normal` x 2, push, first touch of 736 MB	4,061
insert pipeline on a pre-faulted table (bench median of 5)	**4,005**
`lift_scan` canonicalization alone (all roots)	1,921
prefetched insert alone, from precomputed keys (with faults)	2,136
first-touch faults alone (1 GiB, read then write per page)	876
scan self-test, 2 keys (bench median of 5; outside the index build)	**1,488**

The four phase timings come from an instrumented copy with per-row timers, in one process. They include the fault cost of the 736 MB state vector, so they add up to more than the bench state loop.

Two findings from the profile:

1. **The canonicalization is not hidden behind the inserts.** The pipeline costs about the sum of its parts (4.0 s against 1.9 + 2.1 s). The look-ahead overlaps the memory latency, but `lift_scan`'s dependent lifting chain (13 serial stages) and its data-dependent candidate loop still sit on the critical path, between the inserts' own mispredicts.
2. **The state loop is field arithmetic.** It does 11 field products per state (2 in prepare, 3 in Montgomery's trick, 6 in finish), plus three table maps and two normal-basis maps. That arithmetic comes with Montgomery's serial prefix chain over the 83 states of each row.

2. Candidates

Every candidate had to produce byte-identical output: the frozen `stored_states_blake3` / `table_slots_blake3` and index digests on K600, M38 and M10; the same relations, probes and Q0 identities; and rank tables equal to the frozen ones. The self-test's coverage was not to change.

#	Candidate	Proof	Local modelled saving	Status
C9	State roots from 1/l, 1/r and 1/(l+r): 7 field products per state instead of 11	algebraic identity + unique inverses (below)	**1.69 s** (state loop 7.07 to 5.39 s)	**implemented**
C7	Index keys from the linear run finder, 8 roots in lock step	equivalence to lift_scan for every word (below)	**1.46 s** (pipeline 4.00 to 2.55 s)	**implemented**
C6	Bucketed (radix-ordered) insert	plain radix order: **counterexample**; with a left-to-right spill carry: exact (proof below)	**-1.35 s** (slower)	rejected on cost
C8	Build the state loop with `pclmulqdq` enabled so the field products inline	same arithmetic, different inlining	0.0 s	rejected (no saving)
S3	Branch-free bitmap in the self-test (same domain, same three checks)	same predicate, evaluated without early exit	0.06 s (noise)	rejected (no saving)

C9: the state roots from fewer products (proof)

For a stored state with left code l and right code r, the parent's `S3Solver128` computes:

- a = (l + r)^2, p = l r, ps = p^2, c = ps + b;
- the denominator a ps, and inv = (a ps)^-1 (one inversion per row);
- q = p (ps inv), d = (c a)(a inv);
- the trace test on d, then first = q HT(d), and the roots [first, first + q].

When a ps = 0 it falls back to `s3_x_roots_128(gf, b, 1, r)`.

The round-2 loop computes, for a **regular** state (l != r, l != 0, r != 0):

- q' = p $((l + r)^{-1)}2$;
- d' = c $(l^{-1} + r^{-1)}2$.

These are the same field elements:

- q = p ps / (a ps) = p / a = p ((l + r)^-1)2 = q';
- d = c a^2 / (a ps) = c a / ps, and a / ps = (l + r)^2 / (l r)^2 = ((l + r)/(l r))^2 = (1/r + 1/l)^2, so d = d'.

The regular states are exactly the parent's non-zero denominators. Squaring is injective, so a = 0 iff l = r, and GF(2^83) has no zero divisors, so ps = 0 iff l = 0 or r = 0. Every irregular state (the 600 or 608 diagonal relative-0 states per base) takes the parent's own fallback call.

The inverses are field inverses:

- 1/l and 1/r: the inverse of a representative x, squared k times, is the inverse of its k-th Frobenius shift $x^{(2^k)}$. Each one is checked by a product at build time.
- 1/(l + r): Montgomery's trick over four interleaved chains (position mod 4), with one inversion per row. The chains break the parent's serial prefix chain.

A non-zero field element has exactly one inverse, and every field element has exactly one reduced representation. So q', d', the trace test, the half trace, the root order, the `to_normal` images and the pushed states are the parent's, bit for bit. The cost is 7 products per state: p, d, q, first, and 3 for Montgomery's trick.

C7: the linear run finder (proof)

lift_scan's convention: bit i is normal-basis coefficient i, rotl(x, s) = ((x « s) | (x » (N - s))) & mask, and the canonical form is (min_s rotl(x, s), least such s). The words 0 and all-ones get shift 0.

For a nonconstant N-bit word v, let z_1 = !v and z_(j+1) = z_j & rotl(z_j, 1). By induction, bit p of z_j is set iff the j bits p, p-1, ..., p-j+1 (cyclically) are all zero:

- z_(j+1)[p] = z_j[p] & z_j[p-1];
- that is "bits p..p-j+1 zero" and "bits p-1..p-j zero", which is "bits p..p-j zero".

So z_j != 0 iff j <= L, where L is the longest cyclic zero run (1 <= L <= N - 1 for a nonconstant word). The loop stops with z = Z_L, the tops of all longest runs. That is exactly the run set lift_scan's greedy lifting ends with (`lift2/src/lib.rs`: "R = the tops of all longest runs").

The candidates s = N - 1 - p and the choice between them are lift_scan's own code: the least rotation, ties to the smaller shift. That choice is a lexicographic minimum, so it does not depend on the order in which the candidates are visited. Hence the key and the shift equal lift_scan's for every word.

In the eight-lane form, a lane that has reached its Z_L keeps it by a select (no jump) while the other lanes continue. The word 0 starts at z = 0 (an all-ones z would never shrink). The keys are computed once per root, in increasing root order: eight at a time, 32 to 40 roots ahead, with the tail one at a time by the parent's call. Each key's home slot is prefetched. `insert_if_absent` receives the parent's (key, value) sequence, so every slot word is the parent's. In `lift_plant` every linear key is also checked against lift_scan and the selected canonicalizer, with one planted count per root as before.

Tested:

- exhaustively against the full scan on every word of 3 to 20 bits, both the scalar and the eight-lane form;
- exhaustively against lift_scan on eight small two-limb geometries ((5,3), (7,4), (9,5), (12,8), (13,7), (16,8), (18,12) and (20,11), which exercise the limb carry and the wrap);
- against lift_scan on 2^22 random 83-bit words and on about 207,000 structured ones: every single run at every start (including across the wrap), two- and three-way ties, sparse and dense words, and 0 and all-ones;
- against lift_scan on **every stored root of K600, M38 and M10** (29,880,600 + 30,682,720 + 29,880,600 roots, 0 key and 0 shift mismatches). About 27% of those roots have tied longest runs.

C6: bucketed or radix-ordered insert (proof and rejection)

A plain radix or bucket order is not exact. Linear probing with first-wins is order-dependent. Take keys x and y with home h, and z with home h + 1:

- inserted in the order z, x, y, the table holds z at h+1, x at h and y at h+2;
- sorted by home (x, y, z), it holds x at h, y at h+1 and z at h+2.

So the final layout is **not** order-independent, and sorting the inserts by home slot changes `table_slots_blake3`.

An exact bucketed form exists. It does the following:

1. Partition the inserts stably by home-slot range: 2^k contiguous ranges, with each insert's global index kept.
2. Process the ranges left to right. A probe that walks past its range's end becomes a *carry* into the next range. The carry enters that range at its first slot, with its global index.
3. Within a range, merge the range's own inserts and the incoming carries by global index.

Proof. An insert reads and writes only the slots from its entry point forward to where it stops. So range k's slots are touched only by the keys that enter range k: its own keys at their home slots, and carries at its first slot. A key leaves range k (becomes a carry) depending only on range k's state at its arrival. By induction on k, the merged sequence is the parent's sequence restricted to range k, in the parent's order, so range k evolves exactly as in the parent, and so do its outgoing carries. Duplicates are covered: a duplicate has its first occurrence's home, so it follows the same path. A wrap from the last range into range 0 needs a second pass over range 0. None occurred on M38.

Measured: exact on M38 (slot words equal for 2^10, 2^12 and 2^14 buckets, with 195, 783 and 3,138 carries), but slower. The partition alone costs 0.8 to 1.7 s, the bucketed insert costs 2.4 to 5.0 s, and the prefetched insert it would replace costs 2.1 s. The prefetched insert is no longer latency-bound: most of what is left is first-touch and bandwidth, which bucketing does not remove. C6 also needs about 614 MB more memory. **Rejected on cost.**

Not in scope

- **Sampled self-test (round 1's S2).** It is not used, as instructed. The self-test is the parent's exhaustive eight-image loop, unchanged.
- **Rank stage (31% of E2E).** Its code is unchanged and gated identical per seed. Its probe loop also canonicalizes with lift_scan (`KIC_RANK_CANON=lift`, `KIC_TARGET_CANON=lift`), so C7 could carry over there exactly. That is a separate, named follow-up, not part of this round.
- **Other lift_scan variants measured:**
  - lift_scan's own runs, 8 lanes: 1.35 s;
  - linear with 10 jump-free steps: 1.69 s;
  - block-pipelined lift_scan: 3.59 s.

  All are exact, and the chosen lane-linear pipeline beats each of them.

3. What is implemented: a counted derivation

`make_indexbuild2_source.py` follows the parents' `make_*_source.py` pattern:

1. It re-derives the row's producer with the parent's `make_indexbuild_source.py` (verbatim). That script re-derives the closure + index-lift producer `1d328b2e...` through its own verbatim chain and runs its own inverse check.
2. It requires the result to be `bc6a31c7...`.
3. It applies **7 counted replacements**, each of which must occur exactly once:
   - the header comment;
   - two constants;
   - the new functions;
   - the state loop (C9);
   - the compact insert loop (C7);
   - the summary field `index_build_pipeline`, now `sum_inverse_lanes4+linear_canon_lanes8+lookahead32_prefetch+selftest_lanes8`;
   - one appended test module.
4. `check_inverse()` undoes them on every derivation and must get the parent's bytes back.

The derived source is `4008a47a...` (`koblitz_orbit_dlp_fast_online_indexbuild2.rs`), and the binary is `ic_online_indexbuild2_r770`. Both IC binaries are built by one `cargo build` from the parent's lock file. `online/Cargo.toml` drops the grandparent's `[[bin]]` (it is still derived, but no longer an arm) and adds this one.

`restore.py` checks the following on every build:

- the parent protocol `3073d0c5...`;
- side A's producer equals the parent's frozen `bc6a31c7...`, and the grandparent equals `1d328b2e...`;

- both closure headers;
- the 29 files copied byte for byte from the parent (`VERBATIM`), including `arm.py`, `make_schedule.py`, `run_schedule.py`, `check_log_table.py`, `lift2/` and `scripts/host_cpu_record.py`;
- every local pin in `protocol.json`.

`indexbuild2_tests` (7 tests, in the online crate's `cargo test --bins`, which the workflow runs before timing). The parent's state loop and compact insert loop are kept there verbatim (re-indented) as references. The tests cover:

- C9's states, slot words, entries and planted counts against the parent's loops, for widths 1, 2, 9 and 17, every index canonicalizer (selected, lift, lift_plant), folded and planted modes, and compact and base tables;
- `s3_roots_from_inverses` against `S3Solver128::roots` on 200,000 random pairs, including the trace test's `None` branch;
- `sum_inverses_lanes4` against `gf.inv` for every count 0..90;
- `index_keys8` against `index_canonical128` in every mode;
- the linear canonicalizer tests listed under C7.

All 32 tests of the derived binary pass: the parent's 25 (including round 1's 4 `indexbuild_tests`, now run against the round-2 functions) and the 7 new ones (`profile/raw2/cargo-test-online-indexbuild2.txt`).

4. Exactness checks before freezing (cloud-agent VM, correctness only)

`inputs/local_exactness.json` is written by `profile/summarize2.py` from `profile/raw2/`. It records two kinds of check.

Whole-base check (`profile/exactness2_patch.py`, a scratch copy of the round-2 producer). On **K600, M38 and M10**:

- every stored state equals the parent's state loop (verbatim);
- every slot word equals the parent's insert loop (verbatim);
- the linear key and shift equal `lift_scan` on every stored root (0 mismatches on each base);
- 600, 608 and 600 irregular states took the fallback.

Full-gate processes (`arm.py`, verbatim from the parent, run with the frozen protocol's arms). The round-2 producer:

- reproduced the frozen **layout digests** on **K600** (`d8cfe443.../5f53764e...`), **M38** (`f7a8d6b8.../6c08c6c2...`) and **M10** (`c90b501b.../4973cfe5...`);
- reproduced the **Q0 identities** on all three bases:
  - 1,987,852, 1,612,533 and 2,130,165 probes;
  - the frozen relations;
  - scalar 3141592653589793;
- passed `cM38ICX` with 30,682,720 planted checks (2 x 15,341,360);
- on rank seed 1501, gave the **same rank relation digest, per-attempt probe vector and root calls** as the row's producer, with the rank table equal to the frozen verified table.

Smoke of the frozen schedule: O1 and P1 after freezing; see `smoke/README.md`.

5. Frozen design (Tier 1, repeated alternating pairs)

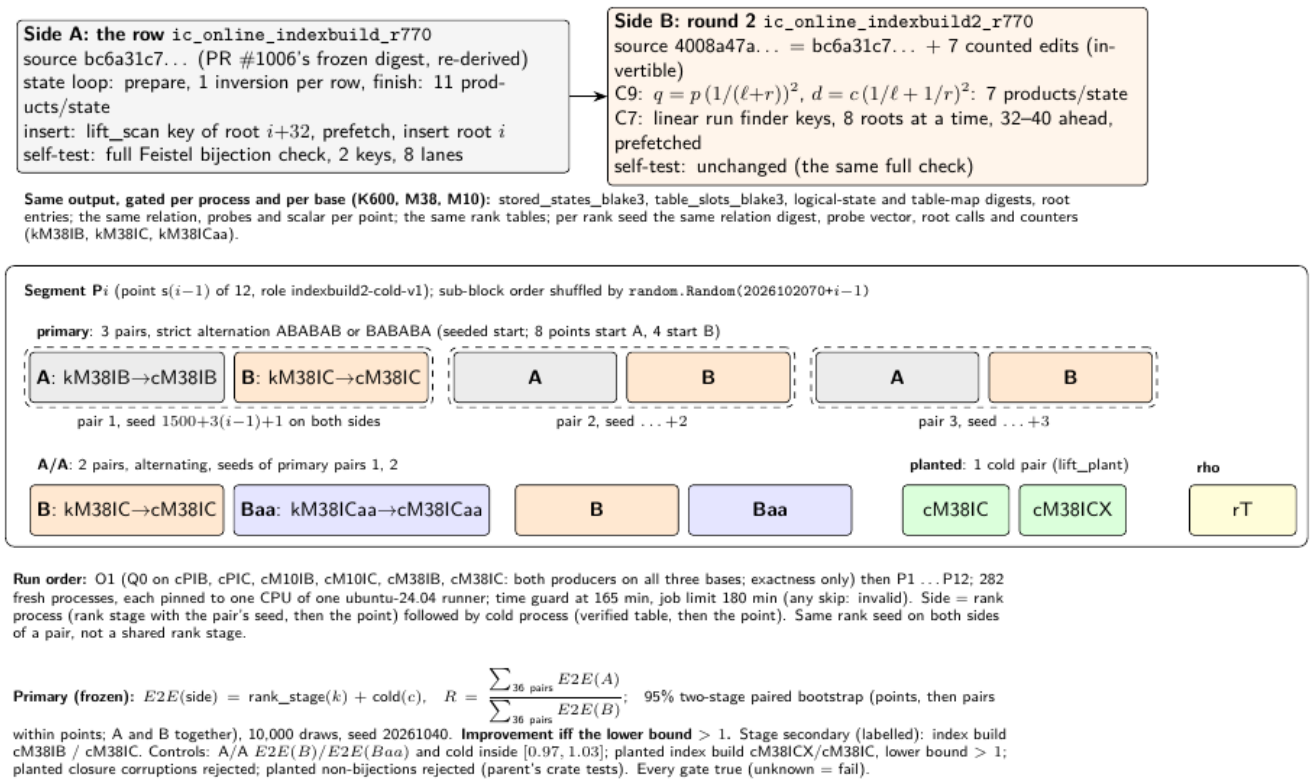


Figure 2: Design (from the frozen protocol and schedule; not a result). Source figures/design.tex, vector figures/design.pdf.

The design is the parent's (itself the Tier 1 template's): one GitHub `ubuntu-24.04` runner, and a fresh process for every run, pinned to one CPU with a clean environment. It keeps:

- `run_schedule.py` (verbatim): CPU record, placement, scheduling counters, cgroup, steal and fault counters, time guard;
- `arm.py` (verbatim);
- the two-stage paired bootstrap, the A/A band, and the hardened fail-closed verifier with only the arm names changed (30 unit tests).

Sides and pairs.

side	processes, in order	binary
A	kM38IB (rank stage, 16 attempts on M38, pair's seed, then the point), then cM38IB (verified table, the point)	the row, `ic_online_indexbuild_r770` (the parent's arms, verbatim)
B	kM38IC (same seed), then cM38IC	round 2, `ic_online_indexbuild2_r770`
Baa	kM38ICaa, then cM38ICaa	A/A copy of B

- **Primary:** 12 points x **3 pairs = 36 primary pairs**. Each point's six sides strictly alternate, ABABAB or BABABA; 8 points start with A and 4 with B. Seed of pair j at point i: $1500 + 3i + j$ (1501-1536, new).
- **A/A:** 2 pairs per point (B against Baa), with the seeds of primary pairs 1 and 2.
- **Planted:** 1 cold pair per point, cM38IC against cM38ICX. In cM38ICX, `lift_plant` checks every linear key against lift_scan and the selected canonicalizer: same layout, slower index build (locally 2.1x).

- **rho:** one rT per point, seed 1040000 + point seed.
- **O1:** Q0 on cPIB, cPIC, cM10IB, cM10IC, cM38IB and cM38IC, so both binaries run on all three bases. This is exactness only.
- In total, **282 processes**. Sub-block order and starts come from `random.Random(2026102070+i)`.

Primary (pre-registered).

- E2E(side) = `timing_ms.rank_stage` (rank process) + `cold_in_process_ms` (cold process).
- R = sum E2E(A) / sum E2E(B) over the 36 pairs.
- 95% two-stage paired bootstrap: points, then pairs within points, with A and B of a pair kept together; 10,000 draws, `random.Random(20261040)`.
- **Improvement iff the lower bound > 1.** Regression iff the upper bound < 1. Otherwise not separated.
- **Invalid** if any gate is false or unknown, if any control fails, or if any row is missing (including time-guard skips).

Stage secondary (labelled): the index-build ratio, cM38IB / cM38IC `timing_ms.index_build`, with the same bootstrap. The run also reports the index build in the rank processes.

Other secondaries (descriptive): geometric mean of the pair ratios; cold, setup, self-test (expected about 1, since the code is unchanged), online, rank stage (expected about 1), CPU time, wall, peak RSS and faults; per point, by pair position and by side order; probe law; S and the gap to rT.

Controls.

- **A/A:** E2E(B)/E2E(Baa) and cold cM38IC/cM38ICaa, each inside [0.97, 1.03] over 24 pairs.
- **Planted timing:** index build cM38ICX/cM38IC, lower 95% bound > 1 over 12 pairs.
- **Planted exactness:**
  - the three closure corruptions rejected before timing (`check_log_table.py`);
  - the parent's planted non-bijections rejected in the crate tests;
  - every cM38ICX checks 2 x 15,341,360 roots and keeps the frozen layout.

Gates. All gates are the parent's and fail closed; an unknown gate counts as a failure:

- every row ran its arm's binary, by name and by digest;
- one index and layout digest set per base on K600, M38 and M10, equal to the frozen one;
- Q0 identities on all six O1 arms;
- one scalar, relation and probe count per point, replayed in the oracle;
- rank tables equal to the frozen tables;
- per rank seed, identical relation digests, probe vectors, root calls and counters across kM38IB, kM38IC and kM38ICaa;
- the probe-law band for kM38IB over 36 seeds (576 attempts);
- placement, CPU identity, schedule identity and order.

Panel and seeds. The panel is 12 new public points, role `indexbuild2-cold-v1`, seeds 0..11 (`make_targets.py`). They are checked against every earlier role of the law, including the parent's `indexbuild-cold-v1`. A role-agnostic x/y scan (`inputs/panel_disjointness.json`) found **0 hits** in all of these:

- origin/main at `5ab7d5bc`;
- 858 other origin branches (1,645,857 changed files);
- the ten scratch n83 worktrees.

The rank seeds 1501-1536 are new (`inputs/seed_disjointness.json`: 85 refs plus the worktrees). The scan's positive control finds the parent's 1401-1436.

6. Power and runtime

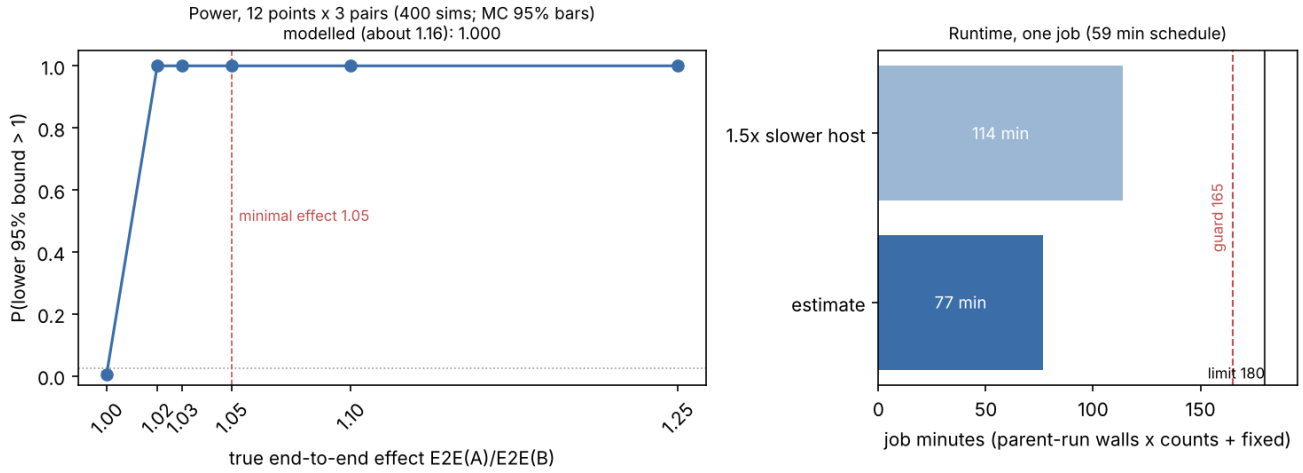


Figure 3: Design-stage power and runtime (not a result), from `analysis/power.json` by `make_figures.py` (svg, pdf).

The prior is the parent's archived run 39383739494 (`make_power_prior.py`, pinned `runs.jsonl` digest `fe0c3659...`):

- the row's 36 primary pairs (kM38IB rank stage; cM38IB cold, index build and self-test);
- its 24 A/A ratios, used as per-process noise factors. This is conservative by about sqrt(2).

Side B is modelled as A minus (1 - f) x index build, with the self-test unchanged. f comes from the local alternating cold processes, 0.729: the round-2 index build at 10.03 s against 13.75 s for the row, three pairs on s000. The smoke of the frozen schedule (`smoke/README.md`) showed a smaller local gap (about 0.87); at that factor the model predicts about 1.07, still above the minimal effect of interest 1.05, where the power is 1.00. In the effect cells, f is set to hit each effect. There are 400 simulations per cell and 500 draws each.

true end-to-end effect	P(lower bound > 1)
1.00 (no gain; type I)	0.005
1.02	1.00
1.03	1.00
1.05 (minimal effect of interest)	**1.00**
1.10	1.00
modelled, about 1.160	**1.00** (median interval hi/lo 1.013)

At the parent run's means, the model predicts the following. These are estimates for sizing the design, not claims:

- end to end about **1.16**;
- cold about 1.25;
- index build about 1.37.

The controls behave as follows in simulation:

- the A/A point estimates are inside the band in 100% of simulations;
- the probe-law gate (576 attempts) passes at the archived law in 100% and rejects a law 1.5x worse in 100%;
- the planted lower bound exceeds 1 in 100%, modelled at 1.5x.

Runtime. The process walls are the parent run's, with side B's reduced by the modelled saving, giving:

- about 287 s per point segment;
- a **59 min** schedule;

- a **job of about 77 min** with 16 min fixed before (restore, build of three binaries, both crates' tests, table checks) and 2 min after;
- **114 min** on a host 1.5x slower.

The time guard is **165 min** and the job limit is **180 min**, so one job fits within the 3 h budget. Any time-guard skip makes the run invalid.

Trigger

The run is the workflow `.github/workflows/n83-indexbuild2.yml` (`workflow_dispatch`, plus a push trigger restricted to branch `claude/n83-indexbuild2` and to this experiment's non-receipt paths). The trigger outcome and the run URL are recorded in Run.

Run

A `workflow_dispatch` call (ref `claude/n83-indexbuild2`) returned **403** ("Resource not accessible by integration"). The workflow therefore also has a push trigger. It is restricted to this branch and to the experiment's non-receipt paths, as in the parent's `n83-indexbuild.yml`. The push that added it (commit `d2d63308`) started the run at the frozen protocol (`5cee1a72...`) and schedule (`f5ac4c17...`). The run is 39464544962. The result will go in its own section, leaving the design text above unchanged.

Files

File	Role
`make_indexbuild2_source.py`	the derivation of side B's producer (7 counted, invertible edits on `bc6a31c7...`)
`protocol.json`, `make_protocol.py`, `schedule.json`	frozen protocol (sha256 `5cee1a72...`), its builder, frozen run order (`f5ac4c17...`)
`restore.py`	restore crypto 770f6a69, derive the three producers, regenerate the closure headers, check every parent, template and local pin
`verify_receipt.py`, `test_verify_receipt.py`	the parent's gates, statistics and fail-closed decision rule with this experiment's arm names (30 unit tests)
`make_targets.py`, `inputs/cold_targets.json`, `inputs/panel_disjointness.json`, `check_seed_disjoint.py`, `inputs/seed_disjointness.json`	the new panel, seeds, and their disjointness records
`profile/`	profiling and exactness patches (scratch copies only), raw records in `raw2/`, summarizer; `inputs/local_profile.json`, `inputs/local_exactness.json`
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39383739494), simulation and result
`arm.py`, `make_schedule.py`, `run_schedule.py`, `check_panel_disjoint.py`, `check_log_table.py`, `make_*_source.py` (other), `online/Cargo.lock`, `lift2/`, `make_closure_base.py`, `verify_closure.py`, `inputs/*` (parent)	the parent's files, byte for byte
`figures/profile.*`, `figures/power.*`, `make_figures.py`, `figures/design.*`	profile and power figures (SVG, PDF, PNG); design diagram (TikZ source, PDF, PNG)
`smoke/`	local correctness smoke of the frozen schedule
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`